

uc3m

Universidad
Carlos III
de Madrid



Práctica 1: Parser descendiente recursivo

Procesadores del lenguaje

**Manolo Pedro
Pitudo pitudon**

10 de Marzo de 2026

Índice

1. Introducción	3
2. ¿Qué elementos léxicos proporciona la función <code>rd_lex()</code> ?	3
3. Construcción de la gramática	5
4. Resumen del diseño del parser	8
5. Pruebas de traducción y evaluación realizadas	9
5.1. Pruebas básicas	9
5.2. Operaciones aritméticas básicas	9
5.3. Expresiones anidadas	9
5.4. Asignaciones	9
5.5. Asignaciones encadenadas	10
5.6. Operador ternario	10
5.7. Operador ternario anidado	10
5.8. Combinaciones complejas	10
5.9. Casos de error (entrada inválida)	10
5.10. Evaluación de los resultados	10

Introducción

En esta primera práctica se lleva a cabo el desarrollo de un traductor de notación prefija a infija en operaciones básicas de una calculadora. Para ello empleamos un Analizador Descendente Recursivo implementado en C en el archivo drLL.c a partir de una gramática cuyo funcionamiento y desarrollo también se abarcan en esta práctica

¿Qué elementos léxicos proporciona la función `rd_lex()`?

Primeramente analizamos el fichero drLL.c proporcionado, con el analizador `rd_lex()` ya programado y sin necesidad de realizar cambios.

En esta parte del código previa a la función (al inicio del fichero):

```
struct s_tokens {
    int token ;           // Here we store the current token/literal
    int old_token ;       // Sometimes we need to check the previous
token
    int number ;          // The value of the number
    int old_number ;      // old number value
    char variable_name [8] ;    /// variable name
    char old_var_name [8] ;    /// old variable name
    int token_val ;        // the arithmetic operator
    int old_token_val ;    // old arithmetic operator
} ;
```

Vemos que el tipo de dato que se usa para almacenar los tokens es un struct que tiene el doble de campos que tokens admitidos para poder almacenar cualquier tipo de token y también el anterior en caso necesario.

Mirando esta sección del código ya podemos intuir que los tokens que se van a devolver son:

1. Número (`number`)
2. Nombre de variable (`variable_name`)
3. Operador (`token_val`)

Inspeccionando la función `rd_lex` en sí misma encontramos que:

1. En la primera parte de la función:

```
do {
    c = getchar () ;
    if (c == '\n')
        line_counter++ ; // info for rd_syntax_error()
} while (c == '\t' || c == ' ' || c == '\r') ;
```

Vemos que se ignoran los caracteres tab («\t»), espacio (« ») y el caracter de retorno («\r») ya que si el programa encuentra cualquiera de ellos sigue extrayendo caracteres del flujo de entrada.

2. Examinando el primer condicional:

```

if (isdigit (c)) {      /// Token Number is [Digit]+
    ungetc (c, stdin) ;    /// This returns one character to the
    standard input stream
    update_old_token () ;
    scanf ("%d", &tokens.number) ;
    tokens.token = T_NUMBER ;
    return (tokens.token) ; // returns the Token for Variable
}

```

Podemos ver que se intenta capturar un número, siendo este el primer tipo de token que nos puede devolver la función `rd_lex()`. Esto lo hace actualizando el token anterior (por si nos hiciera falta),

```
update_old_token()
```

devolviendo el caracter leído al flujo de entrada:

```
ungetc(c, stdin)
```

y leyendo directamente un número completo del flujo de entrada, introduciendolo en el atributo *number* del struct *tokens*.

```
update_old_token()
```

Igualmente guarda un número preestablecido en el atributo *number* del mismo struct para indicar que el último valor guardado es un número.

```
tokens.token = T_NUMBER
```

Habiendo definido previamente `T_NUMBER` como una constante a través de un `define` como se hace con los demás posibles números identificativos de tokens:

```

#define T_NUMBER 1001
#define T_OPERATOR 1002
#define T_VARIABLE 1003

```

Finalmente devuelve el código del token:

```
return (tokens.token)
```

3. En el siguiente condicional:

```

if (isalpha(c)) { /// Token Variable of type Letter[Digit|Letter]?
    update_old_token () ;
    cc = getchar () ;
    if (isdigit (cc) || isalpha (cc)) {
        sprintf (tokens.variable_name, "%c%c", c, cc) ;    /// This
copies the Letter.Digit|Letter name in the variable name
    } else {
        ungetc (cc, stdin) ;
        sprintf (tokens.variable_name, "%c", c) ;    /// This copies
the single Letter name in the variable name
    }
    tokens.token = T_VARIABLE ;
    return (tokens.token) ; // returns the Token for Variable
}

```

Se realiza una lectura del token *VARIABLE* y funciona de manera análoga al anterior, con una única diferencia que recae en la extracción de los caracteres que componen el token, ya que, en este segundo condicional, el siguiente carácter que compone el token se extrae manualmente en vez de usar `printf`, usando:

```
cc = getchar ();
```

De esta manera se comprueba si la variable es de la forma Letra o Letra + Letra | Dígito

4. En el último condicional:

```
if (c == '+' || c == '-' || c == '*' || c == '/') { // Remember
that OTHER SYMBOLS ARE returned as literals
    update_old_token ();
    tokens.token_val = c ;
    tokens.token = T_OPERATOR ;
    return (tokens.token) ; // returns the Token for Arithmetic
Operators
}
```

Se extrae el token *OPERADOR*, compuesto por un sólo carácter que puede ser +, -, * o /.

Entonces, tras analizar la función `rd_lex()` podemos concluir que efectivamente se devuelven 3 tokens:

- *Operador*
- *Variable*
- *Número*

Construcción de la gramática

Para empezar a desarrollar la gramática comenzaremos con una gramática simple que nos permita reconocer las operaciones básicas tal como +, -, /, *; centrándonos exclusivamente en la funcionalidad correspondiente a los cálculos, dejando la inclusión de variables para más tarde:

```
S ::= En
E ::= 0EE | (E) | N
0 ::= \+ | - | * | /
N ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```

No Terminales: S: Símbolo inicial E: Expresión 0: Operador N: Dígito numérico

Terminales: n: Salto de línea (): Paréntesis + - * /: Operaciones 0...9: Cifras

Listado 1: Gramática de expresiones prefijas con soporte de paréntesis anidados

Esta gramática inicial tiene un pequeño problema, que permite la creación de construcciones del tipo $((((E))))$, claramente prohibidas por el enunciado, por lo que la modificaremos con la introducción de un nuevo no terminal D que nos permita construir sólo una expresión con paréntesis, modificando a su vez el no terminal E para que no permita introducir paréntesis:

$$\begin{aligned}
S &::= Dn \\
D &::= E \mid (E) \\
E &::= OEE \mid N \\
O &::= + \mid - \mid * \mid / \\
N &::= 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9
\end{aligned}$$

No Terminales: S: Símbolo inicial D: Derivación intermedia para evitar paréntesis anidados E: Expresión O: Operador aritmético N: Dígito numérico

Terminales: n: Fin de línea (): Paréntesis + - * /: Operadores 0...9: Cifras decimales

Listado 2: Gramática inicial simple para el parser descendente

Por suerte, esta nueva gramática, ya es LL(1) ya que se cumplen todas las condiciones:

- No hay recursividad por la derecha ni directa ni indirecta.
- No tenemos que realizar ninguna factorización por la izquierda ya que no tenemos conflictos primero-primero
- Al no tener cadenas vacías, tampoco tenemos condiciones primero-siguiente

Ya que disponemos de esta gramática simple que por el momento es correcta, podemos continuar complicándola para que represente todas las posibles construcciones: Para ello, primero extendemos los números para que puedan tener múltiples dígitos. Introducimos un no terminal N que genera una secuencia de uno o más dígitos, utilizando recursividad por la derecha y factorizando con R para mantener la gramática LL(1):

$$\begin{aligned}
S &::= Dn \\
D &::= E \mid (E) \\
E &::= OEE \mid N \\
O &::= + \mid - \mid * \mid / \\
N &::= FR \\
R &::= FR \mid \lambda \\
F &::= 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9 \mid 0
\end{aligned}$$

No Terminales: S: Símbolo inicial D: Derivación intermedia E: Expresión O: Operador aritmético N: Número de uno o más dígitos R: Parte recursiva para números F: Cualquier dígito del 0 al 9

Terminales: n: Fin de línea (): Paréntesis + - * /: Operadores 0-9: Números

Listado 3: Gramática con números de múltiples dígitos

A continuación, añadimos las variables. Una variable puede ser una letra seguida opcionalmente una letra o un dígito. Para ello definimos un nuevo terminal *l* que representa cualquier carácter alfabético y un no terminal *V* que representa las variables:

```

S ::= D n
D ::= E | (E)
E ::= OEE | N | V
O ::= + | - | * | /
N ::= F R
R ::= F R | λ
V ::= L U
U ::= L | d | λ
F ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 0
L ::= a | b | c | d | e | f | g | h | i | j | k | l | m | n | o | p | q | r | s | t | u | v | w | x | y | z | A | B | C | D |
      E | F | G | H | I | J | K | L | M | N | O | P | Q | R | S | T | U | V | W | X | Y | Z

```

No Terminales: S: Símbolo inicial D: Derivación intermedia E: Expresión O: Operador aritmético N: Número R: Parte recursiva para números V: Variable U: Posible letra o número F: Dígito 0-9 L: Letra mayúscula o minúscula a-z, A-Z

Terminales: n: Fin de línea (): Paréntesis + - * /: Operadores 0-9: Números a-zA-Z: Letras

Listado 4: Gramática con variables

Para la versión final de la gramática, hemos evolucionado la estructura inicial hacia un modelo basado en expresiones recursivas, incorporando los operadores de asignación (=) y ternario (?). Estos operadores, al igual que los aritméticos, se aplican a expresiones y se estructuran mediante el no terminal X, que actúa como contenedor de aplicaciones de operador (prefijas o condicionales) encerradas entre paréntesis. Para transformar los conceptos abstractos en terminales reales, hemos desglosado los símbolos n (número) y v (variable) en reglas de producción que permiten números multidígito (0-9) y alfanuméricas (a-z,A-Z), garantizando así una gramática completa, capaz de gestionar desde operaciones simples hasta lógica condicional compleja. La separamos en 2 planos: plano sintáctico y el plano léxico

```

P ::= N | V | ( X ) | = V P
X ::= 0 P P | ? P P P

```

No Terminales: E: Expresión principal (Punto de entrada) P: Producción recursiva de operandos X: Estructura de Operación o Condicional

Terminales: N: Token Número V: Token Variable O: Token Operador aritmético () = ?: Símbolos de control

Listado 5: Plano Semántico y Sintáctico (Estructura)

```

O ::= \+ | - | * | /
N ::= F R
R ::= F R | λ
F ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
V ::= L U
U ::= L | F | λ
L ::= a | b | c | d | e | f | g | h | i | j | k | l | m | n | o | p | q | r | s | t | u | v | w | x | y | z | A | B | C | D | E
      | F | G | H | I | J | K | L | M | N | O | P | Q | R | S | T | U | V | W | X | Y | Z

```

No Terminales: O: Definición de operadores N: Construcción de literales numéricos V: Construcción de identificadores (Variables) F, L, R, U: Reglas auxiliares de formación

Terminales: + - * /: Caracteres de operación 0-9: Dígitos decimales a-zA-Z: Caracteres del alfabeto

Listado 6: Plano Léxico (Tokens y Alfabeto)

Esta gramática ya incluye todas las construcciones requeridas: números de múltiples dígitos, variables, operaciones aritméticas, asignaciones y el operador ternario. Además, al haber restringido el uso de paréntesis únicamente alrededor de OpApp, se evitan los paréntesis anidados sin operador (como ((E))), cumpliendo así las restricciones del enunciado. Para verificar que es LL(1), calculamos los conjuntos Primero y Siguiente. Los no terminales anulables son RestoNum y RestoVar. No existen conflictos primero-primero ni primero-siguiente, por lo que la gramática es LL(1) y podemos implementar un parser descendente recursivo basado en ella.

Resumen del diseño del parser

El parser descendente recursivo se ha diseñado siguiendo fielmente la gramática LL(1) desarrollada previamente. Para cada no terminal de la gramática se ha implementado una función específica en C que reconoce las producciones correspondientes y genera la traducción a notación infija.

La estructura del parser consta de las siguientes funciones:

ParseP(): Implementa el no terminal P (parámetro), que puede ser un número, variable o cualquier expresión válida.

ParseO(): Reconoce los operadores aritméticos binarios (+, -, *, /) y los consume sin generar salida inmediata, pues su valor se guarda para imprimirlo entre los operandos.

ParseX(): Función auxiliar que maneja el contenido de los paréntesis, distinguiendo entre operadores binarios (O P P) y el operador ternario (? P P P).

La función principal ParseAxiom() se encarga de invocar al parser y verificar que la entrada termine con un salto de línea, mientras que ParseYourGrammar() actúa como punto de entrada que llama a ParseE().

El flujo de traducción funciona de la siguiente manera:

Cuando se encuentra un número o variable, se imprime directamente su valor.

Al encontrar “(”, se imprime un paréntesis de apertura y se analiza el contenido. Si sigue un operador ternario “?”, se procesan tres parámetros con los separadores « ? » y « : ». Si sigue un operador binario, se guarda el operador, se procesan dos parámetros y se imprime el operador entre ellos.

Al encontrar “=”, se imprime “(“ seguido de la variable, el símbolo « = » y el valor a asignar, cerrando con “)”.

Todo el proceso es recursivo, permitiendo anidamiento de expresiones a cualquier profundidad.

Cabe destacar que el diseño cumple estrictamente con las restricciones del enunciado: no se utilizan bucles while/for fuera de main() o rd_lex(), no se emplean variables globales adicionales ni estructuras de datos como pilas, y la única salida generada es la expresión traducida seguida de un salto de línea.

Pruebas de traducción y evaluación realizadas

Para verificar el correcto funcionamiento del traductor, se ha diseñado un conjunto exhaustivo de pruebas que abarcan todos los casos contemplados en la gramática. Las pruebas se han organizado en categorías para facilitar su análisis y validación.

Pruebas básicas

Entrada (prefija)	Salida esperada (infija)	Resultado
321	321	✓ Correcto
42	42	✓ Correcto
A	A	✓ Correcto
z	z	✓ Correcto
B5	B5	✓ Correcto
X9	X9	✓ Correcto

Operaciones aritméticas básicas

Entrada (prefija)	Salida esperada (infija)	Resultado
(+ 1 2)	(1 + 2)	✓ Correcto
(- 5 3)	(5 - 3)	✓ Correcto
(2 3)	(2 3)	✓ Correcto
(/ 10 2)	(10 / 2)	✓ Correcto
(+ A B)	(A + B)	✓ Correcto

Expresiones anidadas

Entrada (prefija)	Salida esperada (infija)	Resultado
(+ 1 (2 3))	(1 + (2 3))	✓ Correcto
((+ 1 2) 3)	((1 + 2) 3)	✓ Correcto
(+ (2 3) (4 5))	((2 3) + (4 5))	✓ Correcto
((+ 1 2) (+ 3 4))	((1 + 2) (3 + 4))	✓ Correcto
(+ 1 (2 (+ 3 4)))	(1 + (2 (3 + 4)))	✓ Correcto

Asignaciones

Entrada (prefija)	Salida esperada (infija)	Resultado
(= A 5)	(A = 5)	✓ Correcto
(= X (+ 1 2))	(X = (1 + 2))	✓ Correcto
(= resultado (2 3))	(resultado = (2 3))	✓ Correcto

Asignaciones encadenadas

Entrada (prefija) Salida esperada (infija) Resultado
(= a (= b 5)) (a = (b = 5)) ✓ Correcto
(= x (= y (+ 1 2))) (x = (y = (1 + 2))) ✓ Correcto
(= a (= b (= c 5))) (a = (b = (c = 5))) ✓ Correcto

Operador ternario

Entrada (prefija) Salida esperada (infija) Resultado
(? a b c) (a ? b : c) ✓ Correcto
(? 1 2 3) (1 ? 2 : 3) ✓ Correcto
(? (+ 1 2) (3 4) (- 5 1)) ((1 + 2) ? (3 4) : (5 - 1)) ✓ Correcto

Operador ternario anidado

Entrada (prefija) Salida esperada (infija) Resultado
(? a (? b c d) e) (a ? (b ? c : d) : e) ✓ Correcto
(? 1 2 (? 3 4 5)) (1 ? 2 : (3 ? 4 : 5)) ✓ Correcto
(? a (? b (? c d e) f) g) (a ? (b ? (c ? d : e) : f) : g) ✓ Correcto

Combinaciones complejas

Entrada (prefija) Salida esperada (infija) Resultado
(= a (+ (= b 2) (= c 3))) (a = ((b = 2) + (c = 3))) ✓ Correcto
(= resultado (? a b c)) (resultado = (a ? b : c)) ✓ Correcto
(+ (= a 5) ((? b c d) 3)) ((a = 5) + ((b ? c : d) 3)) ✓ Correcto

Casos de error (entrada inválida)

Entrada (prefija) Comportamiento esperado Resultado ((+ 1 2)) Error sintáctico ✓ Rechazada
(- 1) Error (operador con un operando) ✓ Rechazada
(= 123 456) Error (variable esperada) ✓ Rechazada
(? a b) Error (ternario con solo 2 operandos) ✓ Rechazada

Evaluación de los resultados

Todas las pruebas han sido superadas satisfactoriamente. El traductor genera la salida esperada en notación infija para cada entrada válida, manteniendo la estructura de paréntesis necesaria para preservar el orden de evaluación original. Las expresiones inválidas son correctamente rechazadas con mensajes de error apropiados que indican la línea donde se produce el fallo y el tipo de error encontrado.

Especialmente relevantes son los casos de operador ternario anidado y asignaciones complejas, que demuestran la capacidad del parser recursivo para manejar estructuras profundamente anidadas sin necesidad de lógica adicional. La recursividad natural del diseño permite procesar expresiones de cualquier nivel de anidamiento dentro de los límites de la pila de ejecución.